

1. El Operador Ternario (Condicionales en una línea)

Se usa cuando queremos asignar un valor a una variable de forma rápida y directa, reduciendo las líneas de código.

- **Sintaxis:** `variable = valor_si_verdadero if condicion else valor_si_falso`
- **Ejemplo:**

```
n = float(input("Dame un n°: "))
# Si n es positivo se queda igual, si es negativo se le cambia el signo (-)
valor = n if n >= 0 else -n
print("El valor absoluto es:", valor)
```

2. Control de Bucles (break vs continue)

Dentro de un bucle (while o for), estas palabras alteran el flujo del código:

- **break:** **Rompe** el bucle por completo y sale de él.
- **continue:** **Salta** el resto del código de la vuelta actual y vuelve al principio del bucle para la siguiente vuelta.
- **Ejemplo (Bucle con Validación de Entrada):**

```
while True:
    nombre = input("Dame tu Nombre (o 'fin' para salir): ")
    if nombre == "fin":
        break # Cierra el programa
    if nombre == "":
        print("Nombre no válido")
        continue # Vuelve arriba a preguntar el nombre sin cerrar el bucle
    print(f"Hola {nombre}")
```

3. Desempaquetado con Asterisco (*) y Separadores

Cuando imprimes una lista normal, Python añade corchetes y comas ['a', 'b']. El asterisco **rompe los corchetes** y sep elige qué poner entre cada elemento.

- **Sintaxis:** `print(*lista, sep="tu_separador")`
- **Ejemplo**

```
numeros = [10, 20, 30, 40]
```

```
print(*numeros, sep=" - ") # Resultado: 10 - 20 - 30 - 40
```

4. Inversión y Recortes de Listas (Slicing)

Puedes acceder a partes de una lista usando la estructura `lista[principio:fin:salto]`.

- `lista[::-1]`: Invierte por completo el orden de la lista o texto.
- `lista[::2]`: Va saltando de dos en dos.
- **Ejemplo:**

```
letras = ["a", "b", "c", "d", "e"]
```

```
print(*letras[::-1]) # Resultado: e d c b a
```

5. Comprobación de pertenencia (in / not in)

Saber si un elemento está dentro de una colección sin necesidad de recorrerla manualmente.

- **Ejemplo:**
- **3.1. Si queremos buscar una cadena de caracteres (sintaxis que aparece en negrita)**

```
comandos = ["iniciar", "error_404", "detener", "error_conexion"]
```

```
errores = []
```

```
for c in comandos:
```

```
    if "error" in c: # Busca si la palabra "error" está dentro del texto
```

```
        errores.append(c)
```

```
print(errores) # ['error_404', 'error_conexion']
```

6. Diccionarios ({}) como Base de Datos Clave-Valor

A diferencia de las listas que usan índices numéricos (0, 1, 2), los diccionarios guardan información asociada a una palabra clave (clave: valor).

- **Ejemplo (Asignación y lectura dinámica):**

```
productos = ["portátil", "móvil"]
```

```
inventario = {}
```

Llenar el diccionario (en este ejemplo le estaremos dando un valor al elemento en el que estemos en el bucle. Y ese valor lo guardamos en el diccionario. Cuando llamemos luego a ese elemento concreto tendrá el valor que le dimos en la pregunta)

```
for p in productos:
```

```
    inventario[p] = int(input(f"Unidades de {p}: "))
```

Recorrer el diccionario (la variable 'i' toma el valor de la CLAVE)

```
for i in inventario:
```

```
    print(f"Hay {inventario[i]} unidades del producto {i}")
```

7. Métodos Esenciales de Limpieza y Transformación

Los strings en Python tienen funciones integradas listas para usar:

- `.upper()`: Todo a mayúsculas.
- `.lower()`: Todo a minúsculas.
- `.replace("buscar", "reemplazar")`: Sustituye un texto por otro (muy útil para eliminar espacios usando `.replace(" ", "")`).

8. Trocear Textos con `.split()`

Corta un string largo y lo convierte en una lista de palabras. Si dejas el paréntesis vacío (), corta automáticamente usando los espacios en blanco. Lo que pongamos entre paréntesis será desde donde corta.

- **Ejemplo:**

```
config = "servidor_activo puerto_8080 modo_seguro"
lista = config.split() # Genera: ['servidor_activo', 'puerto_8080', 'modo_seguro']
for elemento in lista:
    print(f"Procesando: {elemento}")
```

9. Modularidad (Funciones def)

Las funciones empaquetan un bloque de código bajo un nombre para que puedas reutilizarlo tantas veces como quieras sin repetir líneas.

Uso de return frente a print

- print() solo muestra el texto en la pantalla, pero el programa "olvida" el resultado.
- return **devuelve** el valor final hacia afuera de la función para que puedas guardarlo en una variable y seguir operando con él.
- **Ejemplo Completo (el return es la señal de que ya se cierra la función):**

```
# Definición de la función (Molde)
def limpiar_usuario(nombre_sucio):
    sin_espacios = nombre_sucio.replace(" ", "")
    mayusculas = sin_espacios.upper()
    return mayusculas + "@"

# Uso de la función
usuario_final = limpiar_usuario(" juan perez ")
```

- print(f"El usuario procesado es: {usuario_final}") # Imprime: JUANPEREZ@

10. Coger valor de una lista:

```
lista[variable_recorrer]
```

- **Ejemplo**

```
l = ["T","R","W","A","G","M","Y","F","P","D","X","B","N","J","Z","S","Q","V","H","L","C","K","E"]
DNI = int(input("Dime los dígitos de tu DNI"))
let = DNI%23
print("Tu DNI es", DNI, l[let])
```

11. For con rango

for variable in range(inicio, fin, paso/salto). Habitualmente se pondrá un +1 en el número de fin para incluirlo también.

- **EJEMPLO**

```
i = 0
for i in range (0,121,10):
    v = 5/9 * (i-32)
    print("Grados Farenheit:", i, "|" "Grados celisus:", v)
```

- Siempre se excluye el último valor, si lo queremos coger también:

```
for r in range (i, n+1)
```

12. Ordenar lista:

```
lista.sort()
```

13. Ver la longitud de una cadena de caracteres(String), lista o diccionario:

```
len(texto)
```

14. Contar cuantas veces aparece un elemento:

```
variable.count(variable)
```

- **Ejemplo**

```
vocales = ["a","e","i","o","u"]
palabras = input("Dime una palabra: ")
for vocal in vocales:
    numero = palabras.count(vocal)
```

```
print(f"{vocal} aparece {numero} veces")
```

Cuenta cuántas vocales hay en la palabra que establezcamos

- **Ejemplo - Contar cuantas veces se repite un valor especificado**

```
numeros = [1, 2, 3, 2, 4, 2]
resultado = numeros.count(2)
print(resultado) # Salida: 3
```

- **Ejemplo - Contar un carácter especificado en una cadena de caracteres**

```
texto = "banana"
resultado = texto.count("a")
print(resultado) # Salida: 3
```

15. Operaciones aritméticas con listas

```
vector = [1,2,3]
vecto = [-1,0,2]
lista = [vector[0]*vecto[0]+vector[1]*vecto[1]+vector[2]*vecto[2]]
print(lista)
```

16. Formas de convertir una variable a lista

```
numero = 5
lista = [numero]
print(lista) # Salida: [5]
```

```
-----
texto = "hola"
lista = list(texto)
print(lista) # Salida: ['h', 'o', 'l', 'a']
-----
```

```
# Separar por espacios (por defecto)
frase = "Python es genial"
lista_palabras = frase.split()
print(lista_palabras) # Salida: ['Python', 'es', 'genial']
```

```
# Separar por un carácter específico (ej. una coma)
datos = "1,2,3"
lista_numeros = datos.split(",")
print(lista_numeros) # Salida: ['1', '2', '3']
```

17. Contar a partir de cierta posición

```
frase = str("www.phytonparatodos.com")
cadena = frase
primer = cadena[:10]
segund = frase[10:]
suma = primer+segund
print(primer,segund)
print(suma)
```

18. Pasar a binario

```
numero = input("Dame un número: ")
def binario(numero):
    return int(numero, 2)
print("decimal: ",binario(numero))
```

19. Validar - Comprobar que un valor cumple unas condiciones:

```
contraseña = input("Dame tu contraseña: ")
def validar_contraseña(contraseña):
    valido = False
    valido1 = False
    valido2 = False
    valido3 = False
    if len(contraseña) >= 8:
        valido = True
    for contra in contraseña:
        if contra.isupper(): #Mayúscula
            valido1 = True
        elif contra.islower(): #Minúscula
            valido2 = True
```

```
elif contra.isdigit(): #Comprobar si un valor es numérico. También ahí isdecimal
    valido3 = True
return valido and valido1 and valido2 and valido3
print(validar_contraseña(contraseña))
```

20. Tupla:

Es como una lista, con la diferencia de su sintaxis y que sus valores no se pueden modificar de ninguna forma. Puede tener valores de diferentes tipos.

Tupla con un solo valor: tupla_correcta = (5,)

- **Ejemplo - Acceso**

```
coordenadas = (40.4167, -3.7037)
latitud, longitud = coordenadas
```

```
print(latitud) # Salida: 40.4167
print(longitud) # Salida: -3.7037
```

- **Ejemplo - Convertir a tupla**

ficha1 = tuple(ficha1.split(',')) #Coge un String y lo convierte en una tupla apartir de las comas que encuentre.

TODAS LAS OPERACIONES DE ACCESO SON IGUALES

- **Ejemplo - Mostrar valores en un rango**

```
def mensaje(nombres, p, n):
    for nombre in nombres[p:p+n]:
        print(f"Estimado don/doña {nombre}")
lista_nombres = ("carlos", "alan", "ivan", "adrian", "marta", "martin")
mensaje(lista_nombres, 2, 4)
#De la posición 2-4 cogiendo los de las posiciones medias.
```

21. Tupla de tuplas:

```
lista_nombres = (("carlos","h"), ("alan","h"), ("ivan","h"), ("adrian","h"), ("marta","m"),
("martin","h"))
# Accedemos a cada elemento de forma individual.
```


22. Lista de tuplas

```
lista = [("Sande", "Adrian", "D")]
```

Para llegar a un dato concreto, primero usas el índice de la lista (la fila) y luego el índice de la tupla (la columna).

- **Ejemplo**

```
# Lista de tuplas (representando usuarios: Nombre, Edad, País)
```

```
usuarios = [  
    ("Ana", 25, "España"),  
    ("Luis", 30, "México"),  
    ("Carlos", 22, "Argentina")  
]
```

```
# Acceder al segundo usuario (Luis) y luego a su edad (30)
```

```
print(usuarios[1][1]) # Salida: 30
```

Puedes usar un bucle for para separar automáticamente los elementos de cada tupla en variables independientes en cada vuelta.

- **Ejemplo**

```
for nombre, edad, pais in usuarios:
```

```
    print(f"{nombre} tiene {edad} años y vive en {pais}.")
```

EL RESTO DE OPERACIONES SERÁN IGUALES QUE EN UNA LISTA Y UNA TUPLA

23. Quitar un elemento:

```
lista.remove(variable)
```

24. Hacer un menú:

```
def menu():
```

```
    while True:
```

```
        print(""" Opciones:
```

```
            [1] Añadir elemento a lista
```

```
            [2] Eliminar elemento de lista
```

```
            [3] Enseñar lista
```

```

    [4] Salir")
opcion = int(input('Elija una opcion: '))
if opcion == 1:
    añadir_elemento(list_compra)
elif opcion == 2:
    eliminar_elemento(list_compra)
elif opcion == 3:
    enseñar_lista(list_compra)
elif opcion == 4:
    break
menu()

```

25. Lista de listas

usuarioContrasinal = [["Manuel", "canMorto"], ["Pepe", "usuaya"]]

Python

```
print(usuarioContrasinal[0]) # Salida: ["Manuel", "canMorto"]
```

Si quieres solo la contraseña de Manuel (Fila 0, Columna 1):

Python

```
print(usuarioContrasinal[0][1]) # Salida: "canMorto"
```

Si quieres el nombre de Pepe (Fila 1, Columna 0):

Python

```
print(usuarioContrasinal[1][0]) # Salida: "Pepe"
```

El resto tod igual

SIEMPRE HACER TODO DENTRO DE MAIN CUANDO HAGAMOS TRY.

1. Clases

```

class Coche:
    # El constructor: define los atributos del coche
    def __init__(self, marca, color):
        self.marca = marca
        self.color = color

    # Un método: la acción que puede hacer el coche
    def arrancar(self):
        print(f"El {self.marca} de color {self.color} ha arrancado. ¡Rummm!")

# Creas los objetos (instancias) usando el molde
mi_ferrari = Coche("Ferrari", "Rojo")
mi_toyota = Coche("Toyota", "Azul")

# Usas sus métodos
mi_ferrari.arrancar()
mi_toyota.arrancar()

```

2. Hacer privado un atributo:

```

class CuentaBancaria:
    def __init__(self, titular, saldo_inicial):
        self.titular = titular
        self.__saldo = saldo_inicial # Atributo "protegido". Los 2 guiones es lo
que lo indica

```

```

cuenta = CuentaBancaria("Ana", 1000)
# Python te deja hacer esto (aunque no deberías):
print(cuenta._saldo)

```

3. Acceder y modificar atributos privados:

Getters y Setters

```

def setEstado(self, estado):
    self.__estado = estado

def getEstado(self):

```

```
return self.__estado
```

```
# GETTER: Para ver el valor de forma segura
```

```
# SETTER: Para modificar el valor con validación
```

4. Herencia

```
# Desde el archivo 'usuario' importa la clase 'Usuario'
```

```
from usuario import Usuario
```

5. Pasar información a formato legible

```
class Punto:
```

```
    def __init__(self, x, y):
```

```
        self.x = x
```

```
        self.y = y
```

```
    def toString(self):
```

```
        return f"Punto: x = {self.x}, y = {self.y}\n"
```

```
p = Punto(4, 5)
```

```
# ❌ Si haces esto, Python mostrará algo feo como: <__main__.Punto object at 0x...>
```

```
print(p)
```

```
# Para que funcione, estás obligado a llamarlo así:
```

```
print(p.toString()) # Salida: Punto: x = 4, y = 5
```

6. Coger atributos de una clase heredada

```
class Cilindro(Punto):
```

```
    def __init__(self, x, y, radio, altura):
```

```
        super().__init__(x, y)
```

```
        self.radio = abs(radio) #abs es para sacar automáticamente un valor absoluto
```

```
        self.altura = altura
```

7. Obtener el valor concretado de un metodo

```
class Circulo(Punto):
    def __init__(self, x, y, radio):
        super().__init__(x, y)
        self.radio = abs(radio)

    def obtenerDiametro(self):
        return self.radio * 2

    def calcularPerimetro(self):
        return 2 * math.pi * self.radio

    def calcularArea(self):
        return math.pi * (self.radio ** 2)

    def toString(self):
        cadea = super().toString() + f" \nRadio: {self.radio}"
        return cadea

c = Circulo(4, 5, 3)
print(c.toString())
print(f"Diámetro: {c.obtenerDiametro()}")
print(f"Perímetro: {c.calcularPerimetro():.2f}")
print(f"Área: {c.calcularArea():.2f}")
```

8. Comprobar si una variable es de un tipo

```
def setX(self, x):
    if type(x) == int or type(x) == float:
```

9. if __name__ 'main':

El "Superpoder" de las Variables Ocultas

Cada vez que Python ejecuta un archivo, antes de leer tu código, crea algunas variables ocultas de fondo. Una de ellas se llama `__name__`.

- **Caso 1 (Ejecución Directa):** Si tú abres la terminal y corres el archivo directamente (`python mi_archivo.py`), Python le asigna a `__name__` el valor de `'__main__'`.
- **Caso 2 (Importación):** Si otro archivo hace un `import mi_archivo`, Python le asigna a `__name__` el nombre real del archivo (en este caso, `'mi_archivo'`).

Por lo tanto, la condición `if __name__ == '__main__':` significa: *"Si este archivo fue ejecutado directamente por el usuario, haz lo siguiente..."*.

Un ejemplo práctico para entenderlo

Imagina que creas un archivo con una función útil para calcular el IVA (impuesto) y, además, pones un `print` abajo para probar que funcione.

El archivo `calculadora.py`

Python

```
def calcular_iva(precio):  
    return precio * 0.21
```

Aquí ponemos la condición mágica

```
if __name__ == '__main__':  
    # Esto SOLO se ejecutará si corres "python calculadora.py"  
    print("Probando la función calculadora:")  
    print(calcular_iva(100))
```

¿Qué pasa si creas otro archivo llamado `tienda.py`?

Ahora quieres usar esa misma función en tu negocio:

Python

```
from calculadora import calcular_iva
```

```
print("Bienvenido a la tienda")
```

```
precio_final = calcular_iva(500)
```

```
print(f"El precio con IVA es {precio_final}")
```

El resultado:

- **Si no tuvieras el `if __name__ == '__main__':`:** Al ejecutar `tienda.py`, verías en tu pantalla las pruebas de la calculadora (**Probando la función... 21.0**), lo cual ensuciaría tu programa de la tienda con datos basura.
- **Como Sí lo pusiste:** Al importar el archivo, Python ve que `__name__` es igual a `'calculadora'` (no a `'__main__'`), ignora por completo el bloque de pruebas y solo te permite usar la función limpiamente.

💡 **En resumen:** Sirve como una barrera de seguridad. Todo lo que metas dentro de ese `if` se ejecutará únicamente cuando uses ese archivo específico como el programa principal. Si el archivo es adoptado por otro script, esa parte del código se mantendrá dormida.

10. Comprobar tipo

`if isinstance(a, int) or isinstance(a, float):`

11. property:

Esta línea de código sirve para crear una **propiedad** en Python utilizando la sintaxis clásica. Es el mecanismo que utiliza el lenguaje para implementar el **encapsulamiento** (ocultar datos), permitiéndote controlar cómo se lee y cómo se modifica un atributo desde fuera de la clase.

Su gran magia es que te permite usar **métodos** por detrás (los famosos *getters* y *setters*), pero hacia el mundo exterior se comporta como si fuera una **variable común y corriente**.

El desglose de la línea

La función interna de Python `property()` acepta varios argumentos en un orden específico: `property(fget, fset, fdel, doc)`. En tu caso:

1. **get_a (Getter)**: La función encargada de **leer** y devolver el valor del atributo.
2. **set_a (Setter)**: La función encargada de **modificar** y validar el nuevo valor.
3. **a**: El nombre de la propiedad pública que usarán los demás.

¿Por qué se usa? (Ejemplo práctico)

Imagina que estás programando la clase `Punto` o `Cilindro` que vimos antes y quieres asegurarte de que la coordenada o el radio `a` cumpla ciertas reglas (por ejemplo, que siempre sea un número y que no sea negativo).

1. Definimos la clase con `property`

Python

class Ejemplo:

```
def __init__(self, valor_inicial):
    # Guardamos el valor real en un atributo "privado" con guion bajo
    self._a = valor_inicial

# METODO GETTER (para leer)
def get_a(self):
    print("-> Alguien está leyendo la variable 'a'")
    return self._a

# METODO SETTER (para escribir con validación)
def set_a(self, nuevo_valor):
```



```
print(f"-> Alguien quiere cambiar 'a' por {nuevo_valor}")
if isinstance(nuevo_valor, (int, float)) and nuevo_valor >= 0:
    self._a = nuevo_valor
else:
    print("❌ Error: 'a' debe ser un número positivo")
```

```
# AQUÍ ESTÁ TU LÍNEA EN ACCIÓN:
a = property(get_a, set_a)
```

2. ¿Cómo se usa desde fuera?

Lo genial de la propiedad **a** es que quien use tu clase **no tiene que llamar a funciones**. Escribe código limpio como si fuera una variable normal, pero tus filtros de seguridad se activan automáticamente:

Python

```
obj = Ejemplo(10)
```

```
# Al hacer esto, Python ejecuta 'get_a()' por detrás de forma transparente
```

```
print(obj.a)
```

```
# Salida:
```

```
# -> Alguien está leyendo la variable 'a'
```

```
# 10
```

```
# Al hacer esto, Python ejecuta 'set_a(20)' por detrás
```

```
obj.a = 20
```

```
# Salida: -> Alguien quiere cambiar 'a' por 20
```

```
# Intentamos romper las reglas:
```

```
obj.a = -5
```

```
# Salida:
```

```
# -> Alguien quiere cambiar 'a' por -5
```

```
# ❌ Error: 'a' debe ser un número positivo
```



El estilo moderno: El decorador `@property`

Aunque tu línea `a = property(get_a, set_a)` es perfectamente válida y funciona de maravilla, en el Python moderno (Python 3) casi no se escribe así. Hoy en día se prefieren usar **decoradores** (`@`), porque estéticamente el código queda más ordenado directamente sobre las funciones:

Python

```
class EjemploModerno:
```

```
    def __init__(self, valor):
        self._a = valor
```

```
    @property # Esto reemplaza al get_a
    def a(self):
        return self._a
```

```
    @a.setter # Esto reemplaza al set_a
    def a(self, nuevo_valor):
        if nuevo_valor >= 0:
            self._a = nuevo_valor
```

OTRO EJEMPLO

```
@property
def Radio(self):
    return self._radio

@Radio.setter
def Radio(self, radio):
    self._radio=radio
```

Ambas formas hacen **exactamente lo mismo** bajo el capó de Python. Tu línea de código engancha los métodos de lectura y escritura a la palabra clave `a`.

12. try:

Hacer pruebas (dentro de main)

```
if __name__ == '__main__':  
    try:  
        p = pessoa("Alan","554380238","10")  
    except ValueError as e:  
        print("Error: " + str(e))  
        intentos = 0  
        edade = int(input("Introduce la edad: "))  
        while edade < 0 and intentos < 2:  
            intentos += 1  
            print("Edade non valida")  
            edade = int(input("Introduce un número la edada: "))  
        p = pessoa("Alan","554380238",edade)  
        print(p)  
  
    except TypeError as t:  
        print("Error: "+ str(t))  
  
    finally:  
        print("Fin del programa")
```

13. fichero de error:

```
class DniError(Exception):  
    def __init__(self, mensaxe):  
        super().__init__()  
        self.mensaxe = mensaxe  
    def __str__(self):  
        return "Error "+ str(self.mensaxe)
```

14. isalpha:

Comprobar si un valor es str

15. Errores:

Error especificado:

```
raise DniError(f"La letra no es ta entre las seleccionadas para un DNI")
```

Error de tipo:

```
raise TypeError(f"el tipo de {type(dni)} tiene que ser str")
```

16. Método para escribir un mensaje:

```
def __str__(self):  
    return (f"El nombre es: {self.__nome}, o DNI es: {self.__dni} y la edad es:  
{self.__idade}")
```

17. Match:

```
x = 30
```

```
match x:
```

```
    case 10: print("X é 10")
```

```
    case 20: print("X é 20")
```

```
    case _: print("X non é") # El "else" por defecto
```

- **Para qué sirve:** Para evaluar una variable contra múltiples valores fijos de forma limpia, sin encadenar muchos `elif`.
- **El guion bajo _:** Funciona como un comodín (*wildcard*). Si `x` no es ni 10 ni 20, entra ahí de forma obligatoria (es el equivalente al `else`).

OR:

```
match x:
```

```
    case 10 | 20 | 30:
```

```
        print("X")
```

COMBINADA

```
match x:
```

```
    case 10 if x % 2 == 0:
```

```
        print("X....")
```

```
l = (2, 3, 3)
```

```
match l:
```

```
    case (x, y):
```

```
        print(f"la posicion {x}, {y}")
```

```
    case (x, y, z):
```

```
        print(f"la pososicion es {x},{y} y {z} ")
```

- **Para qué sirve:** Python no está mirando los números, está mirando **la forma** de la tupla `l`.
 - Si la tupla tuviera 2 elementos, entraría en el primer `case` y guardaría esos valores en las variables temporales `x` e `y`.
 - Como tu tupla `l` tiene 3 elementos `(2, 3, 3)`, Python descarta el primer caso, encaja perfectamente con el segundo (`case (x,y,z)`), extrae los tres valores automáticamente y ejecuta ese `print`.

```
persoa = {"nome": "Manuel", "curso": "Dam"}
```

```
match persoa:
```

```
    case {"nome": nome, "curso": curso}:
```

```
        print(f"Nome:{nome},curso:{curso}")
```

- **Para qué sirve:** Sirve para comprobar si un diccionario tiene unas **claves específicas** y, si existen, extrae directamente sus valores en variables listas para usar (`nome` y `curso`).
- Si el diccionario `persoa` no tuviera la clave `"curso"`, el patrón no coincidirá y saltará directamente al caso por defecto `case _:` (donde imprimiría el `"Hostia"`).

18. Pass:

```
class figura:
```

```
    pass
```

```

class Circulo(figura):
    def __init__(self,r):
        self.radio=r
class Rectangulo(figura):
    def __init__(self,ancho,alto):
        self.ancho = ancho
        self.alto = alto

```

```
f = Circulo(15)
```

En Python, no puedes dejar el cuerpo de una función o de una clase completamente en blanco; si lo haces, el programa dará un error de sintaxis (IndentationError).

La palabra clave `pass` es una instrucción nula. No hace absolutamente nada cuando se ejecuta, pero le sirve a Python como un "marcador de posición" para decirle: "Aquí va a ir código en el futuro, de momento déjala pasar".

2. ¿Para qué sirve crear una clase vacía como figura?

Sirve principalmente para aplicar un concepto de POO llamado Abstracción y preparar una Superclase (o clase padre).

Imagina que estás desarrollando un software de dibujo o geometría. Vas a tener círculos, rectángulos y triángulos. Todos ellos son "figuras", pero cada uno calcula su área de forma totalmente distinta.

Creas `class figura`: arriba del todo para agruparlas a todas bajo un mismo árbol genealógico:

Python

La clase padre (vacía por ahora, sirve de molde conceptual)

```

class Figura:
    pass

```

Las clases hijas heredan de Figura

```
class Circulo(Figura):  
    def __init__(self, radio):  
        self.radio = radio
```

```
class Cuadrado(Figura):  
    def __init__(self, lado):  
        self.lado = lado
```

3. ¿Qué ventaja te da esto en tu código?

Aunque ahora mismo esté vacía, te permite hacer cosas muy potentes gracias al Polimorfismo:

A. Validar tipos de datos con isinstance

Puedes comprobar si cualquier objeto de tu programa es una figura geométrica, sin importar si es un círculo o un cuadrado:

Python

```
c1 = Circulo(5)
```

```
# ¿El círculo es un tipo de Figura?
```

```
if isinstance(c1, Figura):
```

```
    print("¡Sí! Es una figura geométrica válida.")
```

B. Crear listas ordenadas

Puedes crear una lista de "figuras" mezcladas y tratarlas a todas por igual en un bucle.



En resumen: `class figura: pass` es un lienzo en blanco. Le estás avisando a Python de que vas a construir una jerarquía de objetos geométricos, permitiéndote empezar a estructurar tu proyecto aunque todavía no hayas programado los detalles de cada figura.

20. Método abstracto

Un **abstractmethod** (método abstracto) es una herramienta de la Programación Orientada a Objetos que sirve para **definir una obligación** para las clases hijas.

Cuando marcas un método como abstracto en una clase padre, le estás diciendo a cualquier clase que herede de ella: *"No sé exactamente cómo vas a hacer esto, pero estás **obligada** a programar tu propia versión de este método; si no lo haces, Python no te dejará crear el objeto"*.

Viene perfecto para complementar la clase **Figura** que acabas de crear.

¿Cómo se usa en Python?

En Python, los métodos abstractos no vienen activos por defecto. Necesitas importar el módulo **abc** (que significa *Abstract Base Classes*).

Mira cómo obligamos a que toda figura geométrica sepa calcular su área:

Python

```
from abc import ABC, abstractmethod
```

1. La clase padre debe heredar de 'ABC' para ser una clase abstracta

```
class Figura(ABC):
```

```
    @abstractmethod
```

```
    def calcular_area(self):
```

```
        """Este método está vacío porque una 'Figura' genérica
        no sabe cómo calcular su área. Cada hija sabrá la suya."""
        pass
```

¿Qué pasa si intentamos usarla?

Caso 1: Intentar crear la figura directamente ❌

Las clases abstractas se vuelven "moldes fantasma". Python **te prohíbe** instanciarlas:

Python


```
# Esto dará un error: TypeError: Can't instantiate abstract class Figura...
mi_figura = Figura()
```

Caso 2: Crear una clase hija y olvidar el método ❌

Si creas la clase **Cuadrado** pero olvidas programar el método obligatorio, Python también bloqueará el programa:

Python

```
class Cuadrado(Figura):
    def __init__(self, lado):
        self.lado = lado
    # ❌ Olvidamos poner 'calcular_area'
```

```
# Esto dará un ERROR al intentar usarlo:
c = Cuadrado(4)
```

Caso 3: Cumplir con la obligación (El caso correcto)

La clase hija "implementa" el método abstracto dándole cuerpo y lógica real:

Python

```
class Cuadrado(Figura):
    def __init__(self, lado):
        self.lado = lado

    # Cumplimos la promesa con el padre
    def calcular_area(self):
        return self.lado * self.lado

class Circulo(Figura):
    def __init__(self, radio):
        self.radio = radio
```

```
# Cumplimos la promesa a nuestra manera
def calcular_area(self):
    return 3.1416 * (self.radio ** 2)

# ¡Ahora todo funciona de maravilla!
c = Cuadrado(4)
print(c.calcular_area()) # Salida: 16
```

¿Para qué sirve en el mundo real?

1. **Para crear contratos de equipo:** Si trabajas con más programadores, usar `@abstractmethod` asegura que nadie olvide programar las funciones clave de un objeto.
2. **Para garantizar el Polimorfismo:** Te asegura al 100% que cualquier objeto que sea una `Figura` va a tener el método `.calcular_area()`. Así puedes hacer bucles seguros como este:

Python

```
mis_figuras = [Cuadrado(4), Circulo(3), Cuadrado(2)]
```

```
# Esto jamás fallará porque el método abstracto garantizó que todos lo tienen
for f in mis_figuras:
    print(f.calcular_area())
```

21. Crear lista vacía en un constructor:

```
self.__xogadores = [None] * num_xogadores
```

22. Sin valor

```
self.partida = None
```

23. Crear fichero:

```
# Definimos el nombre o la ruta del archivo que queremos crear
fichero = "notas.txt"
```

```
# Este es el bloque que crea el fichero
with open(fichero, 'w') as fich:
    fich.write("Esta es la primera línea de mi nuevo archivo.\n")
    fich.write("Esta es la segunda línea.\n")

print(f"El archivo '{fichero}' ha sido creado con éxito.")
```

EJEMPLO DE IMPLEMENTACIÓN DE CÓDIGO EN EL FICHERO

```
class Xestor:
    def __init__(self, ruta_fichero):
        self.listaNotas = []
        # Guardamos la ruta en una variable para poder usarla en cualquier método
        self.ruta = ruta_fichero
        self.cargarNotas(self.ruta)

    def añadirNota(self):
        Nota = input("Añade una nota: ")
        self.listaNotas.append(Nota)
        print("Nota añadida correctamente en memoria.")

    def mostrarNotas(self):
        print("\n--- Tus Notas ---")
        for notas in self.listaNotas:
            print(notas)
        print("-----\n")

    def buscarNotas(self, palabra_clave):
        notas_encontradas = []
        for nota in self.listaNotas:
            if palabra_clave.lower() in nota.lower():
                notas_encontradas.append(nota)

        if notas_encontradas:
```

```

        print(f"\nNotas que conteñen '{palabra_clave}':")
        for nota in notas_encontradas:
            print(nota)
    else:
        print(f"\nNon se atoparon notas que conteñan '{palabra_clave}'.")

def cargarNotas(self, fichero):
    # Corregido: El try envuelve al open para que no se rompa si el archivo no
    existe
    try:
        with open(fichero, 'r') as fich:
            for linea in fich:
                self.listaNotas.append(linea.strip())
            print(f"Se han cargado {len(self.listaNotas)} notas desde el archivo.")
    except FileNotFoundError:
        print(f"El archivo '{fichero}' no existe todavía. Se creará cuando guardes.")

# 🛠 ESTE ES EL BLOQUE QUE CREA EL FICHERO REAL EN TU DISCO
DURO

def guardarNotas(self, fichero):
    with open(fichero, 'w') as fich:
        for nota in self.listaNotas:
            fich.write(nota + '\n')
        print(f"¡Archivo '{fichero}' creado y notas guardadas correctamente!")

#
=====

=====

# CÓMO SE USA TODO JUNTO (El bloque de ejecución)
#
=====

=====

if __name__ == '__main__':

```

1. Creamos el gestor apuntando al archivo que queremos

```
nombre_archivo = "notas.txt"
```

```
mi_bloc = Xestor(nombre_archivo)
```

2. Añadimos un par de notas por teclado

```
mi_bloc.añadirNota()
```

```
mi_bloc.añadirNota()
```

3. Llamamos al método que CREA el fichero "notas.txt" y mete las notas dentro

```
mi_bloc.guardarNotas(nombre_archivo)
```

4. Opcional: Mostramos lo que hay en memoria

```
mi_bloc.mostrarNotas()
```

OTRO EJEMPLO

```
with open("resumo_palabras.txt", "w", encoding="utf-8") as resumo:
```

```
    for palabra, conta in frecuencias.items():
```

```
        resumo.write(f"{palabra}: {conta}\n")
```

```
print("\nResumo gardado en 'resumo_palabras.txt'.")
```

encoding="utf-8": Esto es crucial. Le indica a Python que guarde el archivo usando la codificación UTF-8. Sirve para que las palabras con acentos, eñes o caracteres especiales se guarden correctamente y no aparezcan símbolos extraños (como Ã³) al abrir el bloc de notas.

as resumo: El nombre que le asignas al archivo dentro del código para poder meterle datos.

2. for palabra, conta in frecuencias.items():

Tu variable frecuencias es un diccionario de Python que tiene una estructura parecida a esta: {"hola": 5, "mundo": 3}.

Al usar .items(), el bucle for se desdobra en dos variables a la vez: en cada vuelta, palabra guardará la clave (ej. "hola") y conta guardará el valor o contador (ej. 5).

24. Fichero binario:

```
import pickle
# Abre el archivo en modo "rb" (Read Binary / Lectura Binaria)
try:
    with open("tarefas.dat", "rb") as f:
        # pickle.load traduce los bytes del archivo de vuelta a objetos de Python
        tarefas = pickle.load(f)
except:
    # Si el archivo no existe aún, devuelve una lista vacía para evitar el crash
    tarefas = []

# Abre el archivo en modo "wb" (Write Binary / Escritura Binaria)
with open("tarefas.dat", "wb") as f:
    # pickle.dump transforma los objetos de Python en bytes y los graba en el archivo
    pickle.dump(tarefas, f)
```

VERIFICACIÓN DE EXISTENCIA

```
def cargar_clientes():
    # 1. Pregunta al sistema operativo: "¿Existe físicamente el archivo en el disco duro?"
    if os.path.exists(FICHEIRO):
        # 2. Si existe, lo abre de forma segura en modo lectura binaria ("rb")
        with open(FICHEIRO, "rb") as f:
            # 3. Transforma los bytes del archivo en la lista de clientes original
            return pickle.load(f)

    # 4. Si el archivo NO existe (primera vez que corres el programa),
    # evita el error saltándose el 'with' y devolviendo una lista vacía []
    return []
```

ESCRIBIR EN EL

```
def guardar_clientes(lista):
    # Abre el archivo en modo escritura binaria ("wb")
```

```
# Si el archivo no existía, Python lo crea en este preciso instante
with open(FICHEIRO, "wb") as f:
    # Toma la lista de clientes de la memoria RAM, la convierte en bytes y la graba
    pickle.dump(lista, f)
```

25. Fichero CSV

```
import csv

ficheiro = "produtos.csv"

valor_total = 0
baixo_stock = []

with open(ficheiro, newline="", encoding='utf-8') as f:
    lector = csv.DictReader(f)

    for fila in lector:
        id = fila["id"]
        nome = fila["nome"]
        preco = float(fila["prezo"])
        stock = int(fila["stock"])

        # calcular valor do produto
        valor_total += preco * stock

        # comprobar stock baixo
        if stock < 5:
            baixo_stock.append({
                "id": id,
                "nome": nome,
                "stock": stock
            })

print("Valor total do inventario:", valor_total)
```

```
# guardar ficheiro baixo_stock.csv
with open("baixo_stock.csv", "w", newline="", encoding='utf-8') as f:
    campos = ["id", "nome", "stock"]
    escritor = csv.DictWriter(f, fieldnames=campos)

    escritor.writeheader()
    escritor.writerows(baixo_stock)

print("Ficheiro baixo_stock.csv criado correctamente.")
```